

PRÁCTICA 1

LABORATORIO 1

GIT FUNDAMENTALS

Administración de BBDD

Carlos Belando Sánchez

Profesor Dr. Richard Avilés López
Grado en Ingeniería Informática

Enlace al repositorio de GitHub:



1. ¿Cuál es la diferencia entre Working Directory, Staging Area y Local Repository? Da un ejemplo de un archivo pasando por las tres.

Working Directory: Es el espacio de trabajo local en el sistema de archivos del sistema operativo tal como el usuario lo ve y modifica.

Staging Area (Index): Es una zona intermedia que actúa como preparación; permite registrar selectivamente los cambios que formarán parte de la siguiente instantánea.

Local Repository: Es la base de datos interna (ubicada en .git) que almacena de forma inmutable el historial de commits, objetos y ramas.

Ejemplo: Creamos un archivo con `echo "datos" > test.txt` (entra en Working Directory). Ejecutamos `git add test.txt` (pasa a la Staging Area). Finalmente ejecutamos `git commit -m "docs: add test file"` (se consolida en el Local Repository).

2. Si modificas un archivo pero no haces git add, ¿aparece ese cambio en tu próximo commit? Explica por qué.

No. El comando `git commit` toma exclusivamente una instantánea de lo que está indexado en la Staging Area. Las modificaciones que permanecen únicamente en el Working Directory no son capturadas por el commit hasta que se preparan explícitamente con `git add`.

3. ¿Por qué git status no mostraba las carpetas vacías que creaste en la Parte C? ¿Qué truco usamos para solucionarlo?

Git rastrea el contenido de los archivos y sus estructuras de árbol asociadas mediante objetos blob, no directorios aislados. Si un directorio está completamente vacío, no genera ningún objeto rastreado. Para conservarlo en el repositorio, se utiliza la convención de colocar un archivo marcador vacío denominado `.gitkeep` en su interior.

4. Explica con tus palabras qué es HEAD.

HEAD es un puntero simbólico que referencia la rama o el commit exacto en el que nos encontramos trabajando ahora mismo dentro del repositorio local. Determina qué versión del historial está reflejada en el Working Directory.

5. ¿Qué diferencia hay entre crear una branch con `git switch -c` y crear una carpeta nueva con `mkdir`? ¿Cómo lo comprobamos en la Parte G?

Crear una carpeta con `mkdir` duplica físicamente espacio en el sistema de archivos. Crear una rama con `git switch -c` únicamente añade una referencia ligera (un puntero de 41 bytes) dentro de `.git/refs/heads/` sin duplicar archivos en el disco. Lo comprobamos en la Parte G ejecutando `ls -la`: la estructura del directorio era idéntica y no existía ninguna carpeta con el nombre de la rama. Al cambiar entre ramas con `git switch`, Git reescribió dinámicamente los archivos del directorio de trabajo.

6. Durante el conflicto de la Parte H, ¿qué representaba el contenido entre <<<<<< HEAD y =====? ¿Y entre ===== y >>>>>>?

Entre <<<<<< HEAD y =====: Representa el contenido tal como existía en la rama actual donde estábamos situados (main) al iniciar la fusión.

Entre ===== y >>>>>> fix/readme-subtitle: Representa el cambio entrante proveniente de la rama que intentábamos fusionar.

7. ¿Por qué NO se debe hacer `git commit --amend` sobre un commit que ya se subió con `git push`?

Porque `--amend` genera un objeto de commit completamente nuevo con un hash SHA-1 distinto, destruyendo la referencia anterior y reescribiendo la historia. Si ese commit original ya fue publicado en el

remoto, las copias de los demás colaboradores diferirán, provocando ramas divergentes y errores de sincronización (non-fast-forward) complejos de mitigar en equipo.

8. Si borras por accidente la carpeta .git de tu proyecto, ¿qué se pierde exactamente? ¿Se pierde también el código fuente que está en el disco?

Se destruye la base de datos de Git: el historial de versiones, los commits, las ramas, los stashes, los tags y la vinculación con repositorios remotos. El código fuente actual que reside en el disco (Working Directory) permanece intacto, pero pierde su condición de repositorio y deja de estar versionado.

9. Explica con tus propias palabras la diferencia entre Git y GitHub, sin usar la palabra "nube".

Git es una herramienta de software que se instala y ejecuta localmente en la máquina para gestionar y auditar el historial de cambios del código. GitHub es una plataforma accesible por red que centraliza repositorios de Git remotos para facilitar el trabajo en equipo, la revisión de código por pares (Pull Requests) y el seguimiento de tareas.

10. ¿Por qué no se debe subir un archivo .env con contraseñas reales a un repositorio, aunque el repositorio sea privado?

Porque el modelo de control de versiones almacena cada archivo de forma persistente en los registros del historial. Aunque el repositorio sea privado, cualquier persona con permisos de lectura, claves filtradas, despliegues secundarios o clonaciones locales tendrá acceso indefinido a esas credenciales sensibles. Las buenas prácticas exigen excluirlo con .gitignore y distribuir únicamente plantillas del tipo .env.example.

11. Un compañero te dice: "hice push y ahora GitHub me rechaza el segundo push con 'non-fast-forward'". ¿Qué ha ocurrido probablemente y qué comando ejecutarías primero?

Ha ocurrido que la rama remota en GitHub tiene commits adicionales que el compañero no tiene integrados en su copia local (por ejemplo, cambios añadidos desde la interfaz web o por otro desarrollador). El comando que debe ejecutar primero es:

git pull

Para traer los cambios remotos, fusionarlos con su historial local (resolviendo conflictos si existieran) y, posteriormente, ejecutar git push.

12. ¿Qué tipo de Conventional Commit (feat, fix, docs, test...) usarías para: añadir un índice de rendimiento a una tabla, corregir una restricción mal definida, y actualizar el README?

Añadir un índice de rendimiento a una tabla: perf(db): add index to customer table (o feat(db): si introduce una nueva capacidad en la base de datos). El prefijo perf se reserva específicamente para modificaciones que mejoran el rendimiento del sistema sin alterar la lógica de negocio.

Corregir una restricción mal definida: fix(db): correct foreign key constraint on orders. Se trata de la subsanación de un error o defecto en la base de datos.

Actualizar el README: docs: update setup and usage instructions in README. Cualquier cambio que afecte exclusivamente a archivos de documentación debe llevar el prefijo docs

Capturas de pantalla adjuntas

```
brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git --version
git version 2.55.0.windows.5

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git config --global user.name "Nombre Apellido"

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git config --global user.name "Carlos Belando Sánchez"

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git config --global user.email "cbelando4@alu.ucam.edu"

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git config --global user.name
Carlos Belando Sánchez

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ git config --global user.email
cbelando4@alu.ucam.edu

brech@DESKTOP-RNPN9P5 MINGW64 ~
$ |
```

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$ echo "Draft: customer search notes" > docs/customer-search.md
git add docs/customer-search.md
git commit -m "docs: draft customer search notes"
warning: in the working copy of 'docs/customer-search.md', LF will be replaced by CRLF the next time Git touches it
[feature/customer-search def0166] docs: draft customer search notes
1 file changed, 1 insertion(+)
 create mode 100644 docs/customer-search.md

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git log --oneline --graph --all
* def0166 (HEAD -> feature/customer-search) docs: draft customer search notes
* 1c95f0b (main) docs: fix typo in schema notes
* fd21158 docs: add customer schema notes
* 599b1c2 docs: add initial project documentation

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$ ls docs
customer-schema.md  customer-search.md

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git switch main
Switched to branch 'main'

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ ls docs
customer-schema.md

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git switch feature/customer-search
Switched to branch 'feature/customer-search'

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$ ls docs
customer-schema.md  customer-search.md

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (feature/customer-search)
$
```

Asignatura

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main|MERGING)
$ git add README.md
git commit -m "merge: resolve README title conflict"
[main 66c284d] merge: resolve README title conflict

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git log --oneline --graph --all
* 66c284d (HEAD -> main) merge: resolve README title conflict
|\
| * f1c91c6 (fix/readme-subtitle) docs: rename project title (academic version)
| * dd1cbe6 (fix/readme-title) docs: rename project title (training edition)
|/
| * def0166 (feature/customer-search) docs: draft customer search notes
|/
* 1c95f0b docs: fix typo in schema notes
* fd21158 docs: add customer schema notes
* 599b1c2 docs: add initial project documentation

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ |
```

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (fix/readme-subtitle)
$ git commit -m "docs: rename project title (academic version)"
[fix/readme-subtitle f1c91c6] docs: rename project title (academic version)
1 file changed, 12 insertions(+)

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (fix/readme-subtitle)
$ git switch main
Switched to branch 'main'

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git merge fix/readme-title
Updating 1c95f0b..dd1cbe6
Fast-forward
 README.md | 12 ++++++++
1 file changed, 12 insertions(+)

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git merge fix/readme-subtitle
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main|MERGING)
$ |
```

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (master)
$ git branch -m master main

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ echo "# Database schema notes" > docs/customer-schema.md
```

Asignatura

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        database/migrations/.gitkeep
        database/rollback/.gitkeep
        database/schema/.gitkeep
        database/tests/.gitkeep
        playbooks/.gitkeep
        runbooks/.gitkeep
        scripts/.gitkeep

nothing added to commit but untracked files present (use "git add" to track)
```

```
brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git remote -v
origin https://github.com/Carlos-Belando/oracle-database-lab.git (fetch)
origin https://github.com/Carlos-Belando/oracle-database-lab.git (push)

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (22/22), 2.14 KiB | 439.00 KiB/s, done.
Total 22 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), done.
To https://github.com/Carlos-Belando/oracle-database-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git push -u origin fix/readme-title
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'fix/readme-title' on GitHub by visiting:
remote:   https://github.com/Carlos-Belando/oracle-database-lab/pull/new/fix/readme-title
remote:
To https://github.com/Carlos-Belando/oracle-database-lab.git
 * [new branch]      fix/readme-title -> fix/readme-title
branch 'fix/readme-title' set up to track 'origin/fix/readme-title'.

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git push -u origin fix/readme-subtitle
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'fix/readme-subtitle' on GitHub by visiting:
remote:   https://github.com/Carlos-Belando/oracle-database-lab/pull/new/fix/readme-subtitle
remote:
To https://github.com/Carlos-Belando/oracle-database-lab.git
 * [new branch]      fix/readme-subtitle -> fix/readme-subtitle
branch 'fix/readme-subtitle' set up to track 'origin/fix/readme-subtitle'.

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git push -u origin feature/customer-search
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 403 bytes | 403.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/customer-search' on GitHub by visiting:
remote:   https://github.com/Carlos-Belando/oracle-database-lab/pull/new/feature/customer-search
remote:
To https://github.com/Carlos-Belando/oracle-database-lab.git
 * [new branch]      feature/customer-search -> feature/customer-search
branch 'feature/customer-search' set up to track 'origin/feature/customer-search'.

brech@DESKTOP-RNPN9P5 MINGW64 ~/oracle-database-lab (main)
$ git remote -v
git status
origin https://github.com/Carlos-Belando/oracle-database-lab.git (fetch)
origin https://github.com/Carlos-Belando/oracle-database-lab.git (push)
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```


Asignatura

feature/customer-search had recent pushes 43 minutes ago

Compare & pull request

main

4 Branches

0 Tags

Go to file

Add file

Code

Carlos-Belando merge: resolve README title conflict

66c284d · 1 hour ago

6 Commits

database	docs: add initial project documentation	1 hour ago
docs	docs: fix typo in schema notes	1 hour ago
playbooks	docs: add initial project documentation	1 hour ago
runbooks	docs: add initial project documentation	1 hour ago
scripts	docs: add initial project documentation	1 hour ago
README.md	merge: resolve README title conflict	1 hour ago

README

Oracle Database Lab (Training Edition — Academic Version)

Training repository for Oracle Database administration, testing, change management and Git workflows.

Name: Carlos Belando Sanchez Professor: Richard Aviles Lopez

About

No description, we provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

Carlos-Belando